



Aegis DFF

SECURITY ASSESSMENT REPORT

February 6, 2026

Prepared for:





Contents

1	About CODESPECT	2
2	Disclaimer	2
3	Risk Classification	3
4	Executive Summary	4
5	Audit Summary	5
5.1	Scope - Audited Files	5
5.2	Findings Overview	6
6	System Overview	7
6.1	Contract Architecture	7
6.2	Pool Variants and Position Structure	7
6.3	Redemption Flows	7
6.3.1	Standard Redemption to AG	7
6.3.2	Redemption to Other Token	8
6.3.3	Proportional Multi-Token Redemption	8
6.3.4	ERC-4626 Withdraw	8
6.3.5	Recently Deleted Tokens	8
6.4	Admin Operations	8
6.4.1	Pool Seeding	8
6.4.2	Rebalancing	8
6.4.3	Pool Migration (V1 to V2)	9
6.4.4	Fee Collection	9
6.5	TWAP Price Protection	9
6.6	AegisShares Vesting Mechanism	9
6.7	AegisWithdrawal Atomic Helper	9
7	Issues	10
7.1	[High] First redeemer takes all unclaimed LP fees when collectFees(...) is not called prior to redemption	10
7.2	[Medium] AegisDFF cannot properly handle native ETH pools across multiple flows	11
7.3	[Medium] Liquidity recalculation in withdrawAllEngineShares(...) causes DOS for dismantlePositions(...) when price per share exceeds 1	12
7.4	[Medium] Redeeming 0 assets doesn't transfer the recently deleted tokens	12
7.5	[Medium] dismantlePositions(...) function is vulnerable to price manipulation	13
7.6	[Low] Missing tick spacing adjustment when calculating limit position upper tick during seeding	13
7.7	[Low] The AegisDFF vault is not fully EIP-4626 compliant	14
7.8	[Low] computeTotalAssets() returns incorrect data	14
7.9	[Info] Last AG token fee paid by the user will be stuck in the contract	15
7.10	[Info] Missing deadline parameter in AegisWithdrawal redemption functions to execute at outdated vesting rates	15
7.11	[Info] Redemptions from V2 pools temporarily blocked when AegisEngine pool utilization is high	16
7.12	[Info] V2 pool removal can be grieved via AegisEngine share transfers	16
7.13	[Info] seedPoolFromPosition(...) function doesn't check liquidity position's type	16
7.14	[Best Practice] Checks-Effects-Interactions pattern violation in redeemFromPositions(...)	17
7.15	[Best Practice] Use Ownable2Step instead of Ownable for safer ownership transfers	17
8	Additional Notes	18
9	Protocol Risks	19
10	Evaluation of Provided Documentation	20
11	Test Suite Evaluation	21
11.1	Compilation Output	21
11.2	Tests Output	21
11.3	Notes on the Test Suite	23



1 About CODESPECT

CODESPECT is a specialized smart contract security firm dedicated to ensure the safety, reliability, and success of blockchain projects. Our services include comprehensive smart contract audits, secure design and architecture consultancy, and smart contract development across leading blockchain platforms such as Ethereum (Solidity), Starknet (Cairo), and Solana (Rust).

At CODESPECT, we are committed to build secure, resilient blockchain infrastructures. We provide strategic guidance and technical expertise, working closely with our partners from concept development through deployment. Our team consists of blockchain security experts and seasoned engineers who apply the latest auditing and security methodologies to help prevent exploits and vulnerabilities in your smart contracts.

Smart Contract Auditing: Security is at the core of everything we do at CODESPECT. Our auditors conduct thorough security assessments of smart contracts written in Solidity, Cairo, and Rust, ensuring that they function as intended without vulnerabilities. We specialize in providing tailored security solutions for projects on EVM-compatible chains and Starknet. Our audit process is highly collaborative, keeping clients involved every step of the way to ensure transparency and security. Our team is also dedicated to cutting-edge research, ensuring that we stay ahead of emerging threats.

Secure Design & Architecture Consultancy: At CODESPECT, we believe that secure development begins at the design phase. Our consultancy services offer deep insights into secure smart contract architecture and blockchain system design, helping you build robust, secure, and scalable decentralized applications. Whether you're working with Ethereum, Starknet, or other blockchain platforms, our team helps you navigate the complexity of blockchain development with confidence.

Tailored Cybersecurity Solutions: CODESPECT offers specialized cybersecurity solutions designed to minimize risks associated with traditional attack vectors, such as phishing, social engineering, and Web2 vulnerabilities. Our solutions are crafted to address the unique security needs of blockchain-based applications, reducing exposure to attacks and ensuring that all aspects of the system are fortified.

With a focus on the intersection of security and innovation, CODESPECT strives to be a trusted partner for blockchain projects at every stage of development and for each aspect of security.

2 Disclaimer

Limitations of this Audit: This report is based solely on the materials and documentation provided to CODESPECT for the specific purpose of conducting the security review outlined in the Summary of Audit and Files. The findings presented in this report may not be comprehensive and may not identify all possible vulnerabilities. CODESPECT provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, is entirely at your own risk.

Inherent Risks of Blockchain Technology: Blockchain technology is still evolving and is inherently subject to unknown risks and vulnerabilities. This review focuses exclusively on the smart contract code provided and does not cover the compiler layer, underlying programming language elements beyond the reviewed code, or any other potential security risks that may exist outside of the code itself.

Purpose and Reliance of this Report: This report should not be viewed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. Third parties should not rely on this report for any purpose, including making decisions related to investments or purchases.

Liability Disclaimer: To the maximum extent permitted by law, CODESPECT disclaims all liability for the contents of this report and any related services or products that arise from your use of it. This includes but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services: CODESPECT does not warrant, endorse, or assume responsibility for any third-party products or services mentioned in this report, including any open-source or third-party software, code, libraries, materials, or information that may be linked to, referenced by, or accessible through this report. CODESPECT is not responsible for monitoring any transactions between you and third-party providers. We strongly recommend conducting thorough due diligence and exercising caution when engaging with third-party products or services, just as you would for any other product or service transaction.

Further Recommendations: We advise clients to schedule a re-audit after any significant changes to the codebase to ensure ongoing security and reduce the risk of newly introduced vulnerabilities. Additionally, we recommend implementing a bug bounty program to incentivize external developers and security researchers to identify and disclose potential vulnerabilities safely and responsibly.

Disclaimer of Advice: FOR AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, AND ANY ASSOCIATED SERVICES OR MATERIALS SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Table 1: Risk Classification Matrix based on Likelihood and Impact

3.1 Impact

- **High** - Results in a substantial loss of assets (more than 10%) within the protocol or causes significant disruption to the majority of users.
- **Medium** - Losses affect less than 10% globally or impact only a portion of users, but are still considered unacceptable.
- **Low** - Losses may be inconvenient but are manageable, typically involving issues like griefing attacks that can be easily resolved or minor inefficiencies such as gas costs.

3.2 Likelihood

- **High** - Very likely to occur, either easy to exploit or difficult but highly incentivized.
- **Medium** - Likely only under certain conditions or moderately incentivized.
- **Low** - Unlikely unless specific conditions are met, or there is little-to-no incentive for exploitation.

3.3 Action Required for Severity Levels

- **Critical** - Must be addressed immediately if already deployed.
- **High** - Must be resolved before deployment (or urgently if already deployed).
- **Medium** - It is recommended to fix.
- **Low** - Can be fixed if desired but is not crucial.

In addition to High, Medium, and Low severity levels, CODESPECT utilizes two other categories for findings: **Informational** and **Best Practices**.

- a) **Informational** findings do not pose a direct security risk but provide useful information the audit team wants to communicate formally.
- b) **Best Practices** findings indicate that certain portions of the code deviate from established smart contract development standards.

4 Executive Summary

This document presents the security assessment conducted by CODESPECT for the smart contracts of Aegis. AEGIS DFF is a sophisticated vault system designed to custody AG token liquidity across multiple Uniswap V4 pools with Aegis hooks.

This audit focuses on the Aegis DFF vault system. An ERC4626 vault that manages Uniswap V4 liquidity positions for the AG governance token, paired with bluechip assets, using Aegis dynamic fee hooks.

The audit was performed using:

- a) Manual analysis of the codebase.
- b) Dynamic analysis of smart contracts, execution testing.

CODESPECT found fifteen points of attention, one classified as High, four classified as Medium, three classified as Low, five classified as Informational, and two classified as Best Practices. All of the issues are summarised in Table 2.

Organisation of the document is as follows:

- **Section 5** summarizes the audit.
- **Section 6** describes the functionality of the code in scope.
- **Section 7** presents the issues.
- **Section 8** presents the additional notes raised by auditors.
- **Section 9** presents the risks raised by the auditors.
- **Section 10** discusses the documentation provided by the client for this audit.
- **Section 11** presents the compilation and tests.

Issues found:

Severity	Unresolved	Fixed	Acknowledged
High	0	1	0
Medium	0	4	0
Low	0	2	1
Informational	0	3	2
Best Practices	0	2	0
Total	0	12	3

Table 2: Summary of Unresolved, Fixed, and Acknowledged Issues

5 Audit Summary

Audit Type	Security Review
Project Name	Aegis DFF
Type of Project	ERC4626 Vault
Duration of Engagement	12 Days
Duration of Fix Review Phase	2 Days
Draft Report	February 2, 2026
Final Report	February 6, 2026
Repository	AEGIS_DFF
Commit (Audit)	e8d5ed8a07a09a6d36d0c1401e66957540d90093
Commit (Final)	1589d2ec467877c967229a96a83f369e24854f85
Documentation Assessment	Medium
Test Suite Assessment	Medium
Auditors	talfao , kalogerone , 0xSynthrax

Table 3: Summary of the Audit

5.1 Scope - Audited Files

	Contract	LoC
1	AegisDFF.sol	732
2	AegisShares.sol	130
3	AegisDFFDependencies.sol	66
4	AegisDFFView.sol	80
5	AegisWithdrawal.sol	63
6	AegisToken.sol	23
7	interfaces/IAegisDFF.sol	135
8	interfaces/IAegisShares.sol	47
9	interfaces/IAegisDFFDependencies.sol	53
10	interfaces/IAegisErrors.sol	49
11	interfaces/external/IUniversalRouter.sol	5
12	libraries/AegisDFFPositionLib.sol	562
13	libraries/AegisDFFRedeemLib.sol	335
14	libraries/AegisDFFAssetLib.sol	189
15	libraries/AegisDFFTwapGuard.sol	93
16	libraries/PoolRegistry.sol	72
17	libraries/AegisTypes.sol	16
18	libraries/Constants.sol	7
	Total	2657

5.2 Findings Overview

	Finding	Severity	Update
1	First redeemer takes all unclaimed LP fees when <code>collectFees(...)</code> is not called prior to redemption	High	Fixed
2	AegisDFF cannot properly handle native ETH pools across multiple flows	Medium	Fixed
3	Liquidity recalculation in <code>withdrawAllEngineShares(...)</code> causes DOS for <code>dismantlePositions(...)</code> when price per share exceeds 1	Medium	Fixed
4	Redeeming 0 assets doesn't transfer the recently deleted tokens	Medium	Fixed
5	<code>dismantlePositions(...)</code> function is vulnerable to price manipulation	Medium	Fixed
6	Missing tick spacing adjustment when calculating limit position upper tick during seeding	Low	Fixed
7	The AegisDFF vault is not fully EIP-4626 compliant	Low	Acknowledged
8	<code>computeTotalAssets()</code> returns incorrect data	Low	Fixed
9	Last AG token fee paid by the user will be stuck in the contract	Info	Fixed
10	Missing deadline parameter in AegisWithdrawal redemption functions to execute at out-dated vesting rates	Info	Fixed
11	Redemptions from V2 pools temporarily blocked when AegisEngine pool utilization is high	Info	Acknowledged
12	V2 pool removal can be grieved via AegisEngine share transfers	Info	Fixed
13	<code>seedPoolFromPosition(...)</code> function doesn't check liquidity position's type	Info	Acknowledged
14	Checks-Effects-Interactions pattern violation in <code>redeemFromPositions(...)</code>	Best Practices	Fixed
15	Use <code>Ownable2Step</code> instead of <code>Ownable</code> for safer ownership transfers	Best Practices	Fixed



6 System Overview

The Aegis DFF system is a non-depositable ERC-4626 vault that manages concentrated liquidity positions across multiple Uniswap V4 pools. It operates as a **redemption-only vault**: users holding DFF shares can redeem them for underlying assets, but no new deposits are accepted. The system spans five core contracts and seven supporting libraries, integrating with Uniswap V4's PoolManager and PositionManager, the Aegis Engine lending protocol, and auxiliary vesting and withdrawal helpers.

6.1 Contract Architecture

The system comprises the following in-scope contracts:

- **AegisDFF.sol** — The main ERC-4626 vault contract managing up to 5 Uniswap V4 liquidity pools simultaneously. It holds NFT-based concentrated liquidity positions (base and limit) per pool and serves as the central entry point for all user redemptions and admin operations.
- **AegisShares.sol** — A vesting token (AGS) that allows holders to claim DFF shares over a linear vesting schedule defined by `DURATION_SECONDS`. It includes a 180-day timelocked emergency withdrawal mechanism.
- **AegisWithdrawal.sol** — An atomic helper contract that combines AGS claim and DFF redemption into single transactions, enabling users to go directly from vested AGS to underlying tokens.
- **AegisToken.sol** — The AG token, a SuperchainERC20 with a fixed 1 billion supply.
- **AegisDFFDependencies.sol** — Chain-specific configuration for deterministic CREATE2 deployment.

The vault relies on seven libraries for modular functionality:

- **AegisDFFPositionLib** — Position minting, rebalancing, and liquidity management via PositionManager NFTs.
- **AegisDFFRedeemLib** — Redemption logic including proportional withdrawal, token swaps, and fee application.
- **AegisDFFAssetLib** — Computes `totalAssets()` across all positions and engine shares.
- **AegisDFFTwapGuard** — Price manipulation protection using dual TWAP validation.
- **PoolRegistry** — Registry managing up to 5 active pools with their metadata.
- **AegisTypes** — Shared structs including `PoolData` and `HookVariant`.
- **Constants** — System constants for fees, limits, and TWAP parameters.

6.2 Pool Variants and Position Structure

Each registered pool is stored with a `HookVariant` determining its operational mode:

- **V1 (DFM hooks)** — Positions are managed purely through Uniswap V4 PositionManager NFTs. The vault holds a `baseId` (typically full-range) and a `limitId` (tail/one-sided) position per pool.
- **V2 (Engine hooks)** — In addition to NFT positions, the vault holds shares in the Aegis Engine lending pool. V2 pools are restricted to full-range base positions. Engine shares are tracked via `aegisEngine.balanceOf()`.

Every pool must pair with the AG token as one of its two currencies, enforced at pool registration via the `PoolMissingAsset` check.

6.3 Redemption Flows

Deposits and minting are permanently disabled. The vault supports four redemption paths, each with different fee structures and output formats. The retained fee remains in the vault, increasing the per-share value for remaining holders.:

6.3.1 Standard Redemption to AG

```
function redeem(uint256 shares, address receiver, address owner)
    external returns (uint256 assets);
```

This path carries **0% fee** and is protected by a TWAP guard when no explicit slippage parameter is provided. The flow withdraws proportional liquidity from all pool positions, swaps all non-AG tokens to AG via the Universal Router, and transfers the aggregate AG amount to the receiver.

An overloaded variant accepts a `minAmountOut` parameter, which bypasses the TWAP guard in favour of user-specified slippage protection.



6.3.2 Redemption to Other Token

```
function redeemToOther(uint256 shares, address receiver, address owner,
    Currency currency, uint256 minAmountOut)
    external returns (uint256 amountOut);
```

Applies a **5% fee** (OTHER_FEE_BPS = 500). Withdraws proportionally, converts everything to AG first, then swaps AG to the requested target currency. There is no TWAP guard as the user provides slippage protection.

6.3.3 Proportional Multi-Token Redemption

```
function redeemFromPositions(uint256 shares, address receiver,
    address owner, uint256[] calldata minAmountsOut)
    external returns (Currency[] memory tokens, uint256[] memory amounts);
```

Applies a **2.5% fee** (POSITIONS_FEE_BPS = 250). Returns proportional amounts of *all* underlying tokens (AG plus paired tokens from each pool) without performing swaps. The fee is uniformly applied to three sources: NFT position withdrawals, engine share withdrawals, and idle token balances.

This path burns shares *after* transfers, which deviates from the CEI pattern but is protected by the nonReentrant modifier. Users provide an optional minAmountsOut array for per-token slippage protection, where array ordering matches: AG first, then other tokens in pool registry order.

6.3.4 ERC-4626 Withdraw

```
function withdraw(uint256 assets, address receiver, address owner)
    external returns (uint256 shares);
```

Standard ERC-4626 asset-denominated withdrawal with 0% fee and TWAP protection. Converts the requested asset amount to shares internally. This function is not intended to be used by the Aegis front-end interface, as the normal flow will be through AegisWithdrawal, which uses only the first three redemption flows.

6.3.5 Recently Deleted Tokens

When a pool is removed from the registry, its non-AG token is added to the _recentlyDeletedTokens array. During any redemption, the function _transferRecentlyDeletedTokens distributes the vault's balance of these stranded tokens proportionally to redeemers, applying the same fee structure as the redemption path. The owner can clear entries via removeRecentlyDeletedToken().

6.4 Admin Operations

6.4.1 Pool Seeding

New pools are seeded with an initial one-sided AG limit position via seedPool(pid, agAmount). The function calculates a tick range on the AG side of the current price (at least one full tick spacing away), computes the single-sided liquidity amount, and mints a PositionManager NFT stored as the pool's limitId. TWAP protection is enforced to prevent seeding at manipulated prices.

An alternative seedPoolFromPosition(pid, tokenId) transfers an existing NFT to seed the pool.

6.4.2 Rebalancing

Two rebalancing functions allow the owner to redeploy liquidity:

```
function rebalanceFullRange(PoolId pid, bool consumeAllAg) external;
function rebalanceArbitraryRange(PoolId pid, int24 baseLower,
    int24 baseUpper, int24 limitLower, int24 limitUpper,
    bool consumeAllAg) external;
```

Both are protected by the rebalanceCheck modifier which enforces TWAP guards. The rebalancing flow:

- Pulls all existing positions (burns NFTs, collects fees and principal).
- Calculates budget: consumeAllAg = true uses the entire vault AG balance; false uses only pulled liquidity.
- Mints new base and limit positions at the specified (or auto-calculated) tick ranges.
- For V2 pools, deposits into the Aegis Engine via the router.

V2 pools are restricted to full-range base positions. The limit (tail) position is auto-calculated as a one-sided AG position starting from the current price boundary.



6.4.3 Pool Migration (V1 to V2)

```
function migratePoolFromV1ToV2(PoolId v1PoolId, PoolKey calldata v2PoolKey,
    uint128 minBaseLiquidity, uint128 minLimitLiquidity)
    external returns (uint128 baseLiquidity, uint128 limitLiquidity);
```

Atomically migrates a V1 (DFM) pool to a V2 (Engine) pool. The function validates that both pools use the same currency pair in the same order, removes the V1 pool from the registry, adds the V2 pool, transfers position NFT IDs to the new slot, and triggers a full rebalance. TWAP guards are enforced on both the source and destination pools, and minimum liquidity parameters provide slippage protection.

6.4.4 Fee Collection

The owner can explicitly collect accrued LP fees via `collectFees(pid)`, which executes zero-liquidity decrease operations on both the base and limit positions to harvest fees into the vault's idle balance. This is significant for redemption fairness: without prior fee collection, the first redeemer in a sequence captures all accumulated LP fees through the `DECREASE_LIQUIDITY` mechanism.

6.5 TWAP Price Protection

The system employs dual TWAP validation to guard against price manipulation:

- A **primary TWAP window** (`_twapPeriod`, up to 1200 seconds / 20 minutes).
- An **auxiliary TWAP window** (`_auxTwapPeriod`).
- Separate deviation thresholds for rebalancing (`_rebalanceTwapMaxTicks`) and redemption (`_redeemTwapMaxTicks`), both capped at 200 ticks.

The guard compares the current spot tick against the TWAP tick for each configured period. If the deviation exceeds the allowed threshold, the operation reverts. This protects both user redemptions (preventing sandwich attacks on TWAP-guarded paths) and admin rebalancing (preventing manipulation before liquidity redeployment).

6.6 AegisShares Vesting Mechanism

The `AegisShares` contract implements linear vesting of DFF shares:

```
function claimAegisDff(uint256 amountShares, address to) external;
```

Users burn AGS tokens and receive the corresponding vested amount of DFF. The vesting rate is determined by `DURATION_SECONDS`, with the vested fraction calculated as `min(elapsed / duration, 1)`. The contract includes a separate emergency withdrawal system with a 180-day timelock and a 14-day grace period for execution.

6.7 AegisWithdrawal Atomic Helper

The `AegisWithdrawal` contract provides three atomic operations that combine AGS claiming with DFF redemption:

```
function redeemToAg(uint256 shareAmount, uint256 minAmountOut) external;
function redeemToOther(uint256 shareAmount, Currency currency,
    uint256 minAmountOut) external;
function redeemFromPositions(uint256 shareAmount,
    uint256[] calldata minAmountsOut) external;
```

Each function burns AGS, claims the vested DFF, and immediately redeems through the corresponding `AegisDFF` redemption path in a single transaction.



7 Issues

7.1 [High] First redeemer takes all unclaimed LP fees when collectFees(...) is not called prior to redemption

File(s): `AegisDFF.sol`, `AegisDFFRedeemLib.sol`

Description: All redemption functions in the vault (`redeemFromPositions(...)`, `redeem(...)`, and `redeemToOther(...)`) allow users to redeem their vault shares. During redemption, these functions call `_prepareRedemptionTotals(...)` which invokes `decreaseLiquidity` on Uniswap V4's `PositionManager` to withdraw the user's pro-rata share of liquidity.

```
// All redeem functions ultimately call _prepareRedemptionTotals which withdraws from positions:  
// - redeemFromPositions() -> AegisDFFRedeemLib.redeemFromPositions() -> _prepareRedemptionTotals()  
// - redeem() -> _redeemToAg() -> AegisDFFRedeemLib.redeemToAsset() -> _prepareRedemptionTotals()  
// - redeemToOther() -> AegisDFFRedeemLib.redeemToOther() -> _prepareRedemptionTotals()
```

However, when Uniswap V4's `DECREASE_LIQUIDITY` action is executed, it returns ALL accumulated LP fee for the position to the caller, not just a proportional share. This means the first user to redeem collects 100% of all LP fees that have accrued since the last fee collection, while subsequent redeemers receive zero fees.

Impact: A first redeemer extracts all the LP fees, which should be distributed to all shareholders.

Recommendation(s): Call `collectFees(...)` at the beginning of all redemption functions to capture LP fees into the vault's idle balance before any withdrawal. This ensures fees are distributed proportionally to all redeemers via the `idleShare` calculation rather than going entirely to the first redeemer.

Status: Fixed

Update from Aegis: Fixed: `85a52180b40b83f2e8b8ebc4c5fc3827ed61b8f4`

7.2 [Medium] AegisDFF cannot properly handle native ETH pools across multiple flows

File(s): [AegisDFFRedeemLib.sol](#), [AegisDFF.sol](#)

Description: The AegisDFF vault has multiple issues when handling pools that use native ETH as one of the currencies:

1. Standard redeem(...) reverts with native ETH pools The `_swapToAG(...)` function executes swaps via the Universal Router but does not send ETH value with the call:

```
function _swapToAG(...) private returns (uint256 amountOut) {
    // ...
    uint256 balanceBefore = outputCurrency.balanceOfSelf();
    // @audit Native ETH not sent to router - reverts
    ctx.universalRouter.execute(commands, inputs);
    amountOut = outputCurrency.balanceOfSelf() - balanceBefore;
}
```

When the input currency is native ETH, the transaction reverts because no `msg.value` is passed.

2. removePool(...) does not track native ETH in recently deleted tokens When a pool is removed, the `_addRecentlyDeletedToken(...)` function explicitly skips native ETH:

```
function _addRecentlyDeletedToken(Currency token) private {
    // @audit Native ETH is skipped here
    if (token.isAddressZero() || !_isRecentlyDeletedToken(token)) {
        return;
    }
    _recentlyDeletedTokens.push(token);
}
```

This means when a native ETH pool is removed, the ETH remaining in the vault is not tracked in `_recentlyDeletedTokens` and users cannot claim their proportional share during redemption.

Impact:

- The standard `redeem(...)` function is completely broken for vaults containing native ETH pools;
- When native ETH pools are removed, the ETH becomes stuck in the vault with no mechanism for users to claim their proportional share;

Recommendation(s):

- a. Pass ETH value when calling `universalRouter.execute(...)`;
- b. Consider adding ETH as a custom address to the `_recentlyDeletedTokens` array and also update according to that `_transferRecentlyDeletedToken` function as well;

Status: Fixed

Update from Aegis: Fixed: [7d155c6815a047bedc90da26fff644cb5985ce47](#)

7.3 [Medium] Liquidity recalculation in `withdrawAllEngineShares(...)` causes DOS for `dismantlePositions(...)` when price per share exceeds 1

File(s): `AegisDFFPositionLib.sol`

Description: The `withdrawAllEngineShares(...)` function is invoked during `dismantlePositions(...)` to remove all engine shares from a V2 pool and redeem the underlying value. However, while the function converts vault shares into "liquidity", the Aegis Engine expects *shares* to be provided during withdrawal. This logic works correctly only when the price per share equals WAD (i.e., $1e18$). If the share price increases due to accumulated fees or interest, the recalculation of redeemable liquidity can exceed the actual share balance, causing the withdrawal to revert.

```
function withdrawAllEngineShares(...) internal {
    uint256 vaultShares = engine.balanceOf(address(this), shareId);
    // ...
    uint256 totalShares = AegisStateLibrary.getTotalSupply(engine, shareId);
    uint256 equityLWad = AegisStateLibrary.getMarketEquity(engine, pid);

    // @audit-issue This recalculation leads to a revert when price per share > 1
    uint256 liquidityToRedeem =
        FullMath.mulDiv(vaultShares, equityLWad, totalShares * 1e18);
}
```

As a result, the computed `liquidityToRedeem` may exceed the amount implicitly represented by `vaultShares`, triggering a revert inside the Aegis Engine.

Impact: The `dismantlePositions(...)` function becomes permanently DoS'd for pools where the engine has accrued value beyond the initial deposit. This prevents the owner from successfully closing V2 (engine-backed) pools.

Recommendation(s): Avoid converting shares into liquidity during withdrawal. Instead, pass and redeem `vaultShares` directly, as this is the unit expected by the Aegis Engine and remains consistent regardless of share price appreciation.

Status: Fixed

Update from Aegis: fixed: `d572b9019c967c108c1a57c5a3ebb83e295002a2`

7.4 [Medium] Redeeming 0 assets doesn't transfer the recently deleted tokens

File(s): `AegisDFF.sol`

Description: The `redeem(...)` and `withdraw(...)` functions that call the `_redeemToAg(...)` function don't transfer the recently deleted tokens to the user if the **withdrawn assets** are 0:

```
function _redeemToAg(uint256 shares, address receiver, address owner) internal returns (uint256 assets) {
    // ...
    if (assets == 0) {
        return 0;
    }

    agToken.transfer(receiver, assets);

    // Transfer recently deleted tokens proportionally (no fee for redeem to AG)
    _transferRecentlyDeletedTokens(receiver, shares, supply, netBps);

    emit Redeemed(owner, receiver, shares, Currency.wrap(asset()), assets);

    return assets;
}
```

For example, if all the pools get removed and users should only receive the recently deleted tokens for their redemptions, calling these functions will burn their shares and they will receive nothing. This is handled properly on the other redeem functions.

Impact: Users don't receive their portion of the recently deleted tokens if their withdrawn assets are 0.

Recommendation(s): Consider moving the `_transferRecentlyDeletedTokens(...)` call before the `assets == 0` check.

Status: Fixed

Update from Aegis: Fixed: `bc1b9ef480fac4f8ba6fad91c0542c953e5a7302`

7.5 [Medium] `dismantlePositions(...)` function is vulnerable to price manipulation

File(s): `AegisDFF.sol`

Description: The `dismantlePositions(...)` function withdraws liquidity from Uniswap V4 positions owned by AegisDFF and returns the funds to the vault's idle balances. Dismantling these positions is a prerequisite for removing a pool from the AegisDFF vault. However, the function does not enforce any TWAP-based rebalancing or price validation checks. As a result, a malicious actor could temporarily manipulate the pool's spot price prior to execution, causing the vault to withdraw liquidity at an unfavourable and unintended token ratio from Uniswap V4.

Impact: The vault may exit the position using a manipulated price, leading to an imbalanced withdrawal.

Recommendation(s): Consider enforcing a TWAP-based price validation check within the `dismantlePositions(...)`.

Status: Fixed

Update from Aegis: fixed: `85a52180b40b83f2e8b8ebc4c5fc3827ed61b8f4`

7.6 [Low] Missing tick spacing adjustment when calculating limit position upper tick during seeding

File(s): `AegisDFF.sol`

Description: The `seedPool(...)` function creates one-sided limit positions to provide liquidity for the AG token. When AG is token1 (not token0), the function calculates the position's tick range to be below the current price.

```
if (agIsToken0) {
    // AG is token0, position should be above current price
    tickLower = AegisDFFPositionLib.alignTickToSpacing(tick, tickSpacing, true) + tickSpacing;
    tickUpper = fullRangeUpper;
} else {
    // AG is token1, position should be below current price
    tickLower = fullRangeLower;
    tickUpper = AegisDFFPositionLib.alignTickToSpacing(tick, tickSpacing, false);
}
```

For the token0 case, the code correctly adds `+ tickSpacing` to ensure the position starts one tick spacing above the aligned tick. However, for the token1 case, there is no corresponding `- tickSpacing` adjustment. This inconsistency means the limit position's upper tick may overlap with the current tick, potentially placing the position partially in-range rather than fully out-of-range.

Impact: The limit position may be partially in-range when AG is token1, which could result in immediate conversion of some AG tokens during the seeding process, rather than the intended behaviour of creating a purely one-sided position.

Recommendation(s): Add `- tickSpacing` adjustment for consistency:

```
tickUpper = AegisDFFPositionLib.alignTickToSpacing(tick, tickSpacing, false) - tickSpacing;
```

Status: Fixed

Update from Aegis: fixed: `e7624a71838646bd66ee488fe7f4353b9b15d0a2`



7.7 [Low] The AegisDFF vault is not fully EIP-4626 compliant

File(s): [AegisDFF.sol](#)

Description: The AegisDFF vault has the following features which are not EIP-4626 compliant:

- The `maxMint(...)` and `maxDeposit(...)` functions must not revert;
- The `withdraw(...)` function must revert if all of assets cannot be withdrawn (due to withdrawal limit being reached, slippage, the owner not having enough shares, etc);
- The `withdraw(...)` function should round up to calculate the shares required from the assets;
- The `redeem(...)` and `withdraw(...)` functions must emit an event named `Withdraw` with the inputs of (sender, receiver, owner, assets, shares);

Impact: Other contracts interacting with the AegisDFF vault will expect different behavior than the actual one.

Recommendation(s): Consider implementing the following changes:

- The functions should return `0` instead of reverting;
- The `withdraw(...)` function should revert if user's shares can't match the requested assets amount;
- The `mulDivUp` function should be used to calculate the shares from the assets;
- The withdrawing functions should implement the event mentioned in the report;

Status: Acknowledged

Update from Aegis: We are aware, we did everything in our power to keep it as compliant. Events were removed for contract size requirements but will be readded if possible. If not the `AegisWithdraw` will have events to show redemptions.

Update from CODESPECT: Leaving as Acknowledged

Update from Aegis: Fixed all except 2: [320eae4292cba1bd40d8f33501e667d4c83363c0](#) We don't want to potentially hinder user experience from withdrawals failing. We are not planning to use the standard `redeem` and `withdraw` function in our UI but rather our custom functions in `AegisWithdrawal` that will allow us to more accurately portray withdrawals via static calls and ensure safe withdrawals from slippage protection.

7.8 [Low] `computeTotalAssets()` returns incorrect data

Description: The function `computeTotalAssets(...)` is responsible for returning the AG token value of all assets owned by AegisDFF. It has two issues that result in incorrect values being returned. However, the function is intended for view purposes only so the impact is limited:

1. The function does not include outstanding Uniswap V4 claimable fees:

- Uniswap V4 positions accrue fees on each swap. These fees are collected before each redeem (if the fix for the CODESPECT high-severity issue is applied), meaning users will receive a share of the outstanding fees. However, these values are not included in the total assets owned by the vault;

2. `_v2LiquidityValue(...)` considers only liquidity present in the Engine-owned position:

- The function `_v2LiquidityValue(...)` leads to incorrect asset calculations because it only considers liquidity currently owned by the Engine in the Uniswap pool. In practice, liquidity is withdrawn when assets are borrowed from the Engine, and accrued interest and fees are also taken into account. The function should instead account for the equity owned by AegisDFF;

Impact: Incorrect data returned by this view function can lead to improperly configured slippage protection and may result in losses for users. Additionally, the front end may display inaccurate information.

Recommendation(s): Consider updating the logic to account for outstanding fees and to correctly evaluate the assets and equity owned within AegisEngine.

Status: Fixed

Update from Aegis: Fix: [fc52a9b9f688522bc97f572f8c57469734baba9c](#)



7.9 [Info] Last AG token fee paid by the user will be stuck in the contract

File(s): [AegisDFF.sol](#)

Description: The `emergencyWithdraw(...)` function in the AegisDFF contract is designed to rescue ERC20 tokens that are stuck in the contract. However, it explicitly blocks the withdrawal of the AG token. This creates a scenario where AG tokens that accumulate in the vault through redemption fees cannot be recovered via `emergencyWithdraw(...)`. This can happen if the last remaining shareholder redeems their position via `redeemFromPositions(...)` or `redeemToOther(...)`, which will leave the fees in the contract.

Impact: AG tokens accumulated from the last redemption fee will be stuck in the contract.

Recommendation(s): Consider removing the token restrictions from `emergencyWithdraw(...)` function when there is no share supply.

Status: Fixed

Update from Aegis: Fair point, I never see this arising realistically since there will always be a depositor who will forget, however even if we do run into this circumstance we can should be able to keep this to dust amounts of the AG token.

7.10 [Info] Missing deadline parameter in AegisWithdrawal redemption functions to execute at outdated vesting rates

File(s): [AegisWithdrawal.sol](#)

Description: The AegisWithdrawal contract provides helper functions (`redeemToAg(...)`, `redeemToOther(...)`, `redeemFromPositions(...)`) that burn vested AegisShares and redeem the resulting DFF tokens via the vault. These functions accept a `minAmountOut` for slippage protection but do not accept a deadline parameter.

```
function redeemToAg(
    uint256 shareAmount,
    uint256 minAmountOut
) external nonReentrant returns (uint256 agOut, uint256 dffClaimed) {
    dffClaimed = _claimVestedDff(msg.sender, shareAmount);
    // @audit No deadline - transaction can execute far in the future
    agOut = aegisDff.redeem(
        dffClaimed, msg.sender, address(this), minAmountOut
    );
}
```

The `AegisShares.claimAegisDff(...)` calculates the DFF amount based on the current `redeemablePercent()`, which increases linearly with time. If a transaction is stuck in the mempool, it will execute at a higher vesting percentage than the user intended. The `minAmountOut` was set based on the original vesting rate, so it no longer accurately reflects slippage protection.

Impact: A user submitting a redemption transaction may receive a different amount of DFF than expected if the transaction is delayed. Because `redeemablePercent()` increases over time, a delayed transaction claims more DFF from the AegisShares contract, which could disadvantage other shareholders who expected the original vesting schedule.

Recommendation(s): Add a deadline parameter to all three redemption functions and revert if `block.timestamp > deadline`.

Status: Fixed

Update from Aegis: These are the difference of seconds even minutes, and `minAmountOut` is going to have a .3% difference under intended output. This function works properly and deadline is really not necessary.

Update from CODESPECT: Will decrease the severity and leave as acknowledged

Update from Aegis: Added deadline: [5b86647769ae04e8d5e2a7e4e769067b164c22ed](#)



7.11 [Info] Redemptions from V2 pools temporarily blocked when AegisEngine pool utilization is high

File(s): AegisDFFRedeemLib.sol

Description: The `_withdrawFromPool(...)` function redeems the vault's proportional AegisEngine shares during user redemptions from V2 pools. It calculates the shares to redeem and calls `AE_MODIFY_LIQUIDITY` with a negative delta via the AegisRouter. Inside the AegisEngine, `modifyLiquidity(...)` converts the shares to a liquidity delta and calls `POOL_MANAGER.modifyLiquidity(...)` to remove full-range liquidity. However, if the pool has high utilization (borrowers have consumed most of the available liquidity), the Uniswap V4 PoolManager will revert because the requested liquidity removal exceeds what is physically available in the full-range position. The AegisEngine does not perform a pre-check against available (non-borrowed) liquidity before attempting the removal.

Impact: When AegisEngine pool utilisation is high, all AegisDFF redemption paths will revert for V2 pools. Users are temporarily unable to withdraw their funds until borrowers repay or utilisation decreases. The utilisation cap of 95% means up to 95% of the pool's liquidity can be borrowed, leaving only 5% available for lender withdrawals.

Recommendation(s): Document this as expected behaviour and inform users that V2 pool redemptions may be temporarily delayed during high utilisation periods.

Status: Acknowledged

Update from Aegis: This is known and can be adjusted in Aegis Engine if needed.

Update from CODESPECT: Based on the internal discussion, it was agreed that it should be documented. I

7.12 [Info] V2 pool removal can be grieved via AegisEngine share transfers

Description: The `_removePool(...)` function checks whether the vault holds any AegisEngine shares before allowing the removal of a V2 pool:

```
function _removePool(PoolId pid) private returns (uint8 idx) {
    AegisTypes.PoolData storage slot = _poolRegistry.getData(pid);
    if (slot.variant == AegisTypes.HookVariant.V2) {
        (uint256 shareBalance,) = AegisDFFAssetLib.v2ShareInfo(pid, aegisEngine);
        // @audit-issue This check can be grieved by transferring shares TO the vault
        if (shareBalance != 0) revert ActivePositions();
    }
    idx = _poolRegistry.remove(pid);
}
```

The check assumes that the vault will only hold AegisEngine shares as a result of legitimate protocol operations. However, AegisEngine shares are ERC6909 tokens and can be transferred freely. An attacker can block V2 pool removal by directly transferring a non-zero amount of engine shares to the vault. As a result, the vault temporarily holds engine shares, causing `_removePool(...)` to revert with `ActivePositions()`.

Impact: An attacker can temporarily deny service to the `removePool()` function for any V2 pool at minimal cost. While the owner can recover by calling `dismantlePositions(...)` to clear the shares, the attack can be repeated indefinitely, resulting in a persistent grieving vector.

Recommendation(s): When removing a pool, consider batching the operation with `dismantlePositions(...)` or otherwise ensuring that externally transferred engine shares cannot block pool removal.

Status: Fixed

Update from Aegis: Fixed: 550427de5f46d525f52b7a43aa843a4d0e53900b

7.13 [Info] seedPoolFromPosition(...) function doesn't check liquidity position's type

File(s): AegisDFF.sol

Description: The `seedPoolFromPosition(...)` function accepts an existing Uniswap V4 NFT liquidity position and stores it in the vault's `limitId` slot without verifying whether the position is one-sided or balanced. While there is no technical impact, `limitId` positions are expected to be one-sided AG-token positions from the `seedPool()` function. This creates potential confusion about position classification without causing functional issues.

Impact: Position classification inconsistency.

Recommendation(s): Consider checking the position's type in `seedPoolFromPosition(...)`.

Status: Acknowledged

Update from Aegis: This could be the case but would be fixed upon rebalances. This is done to have additional confidence for the first three weeks when we might have external liquidity providers involved and it would be more comfortable for them to see safety mechanisms they understand such as standard timelock / safe rather than AG token in AegisDFF.

Update from CODESPECT: We are going to mark as acknowledged.



7.14 [Best Practice] Checks-Effects-Interactions pattern violation in `redeemFromPositions(...)`

File(s): `AegisDFF.sol`

Description: The `redeemFromPositions(...)` function burns the user's shares after executing the redemption logic, which can involve the native ETH transfer if one of the managed pools contains native ETH.

```
(tokens, amounts) = AegisDFFRedeemLib.redeemFromPositions(ctx, _poolRegistry); // @audit can contain .call if native
→ ETH in the pools
// ... slippage checks ...
_burn(ctx.owner, ctx.shares); // @audit burn AFTER low level call
```

This violates the Checks-Effects-Interactions (CEI) pattern, where state changes (effects) should occur before external calls (interactions). While the `nonReentrant` modifier is present and prevents direct reentrancy, following CEI is a defence-in-depth best practice.

Impact: No direct exploit identified due to `nonReentrant` modifier protection.

Recommendation(s): Move `_burn(ctx.owner, ctx.shares)` before the `AegisDFFRedeemLib.redeemFromPositions(...)` call to follow CEI pattern.

Status: Fixed

Update from Aegis: Fixed: [660471d90aa4aa052276651e4557facf70bbe6e4](#)

7.15 [Best Practice] Use `Ownable2Step` instead of `Ownable` for safer ownership transfers

File(s): `AegisDFF.sol`

Description: The `AegisDFF` contract inherits from OpenZeppelin's `Ownable` for access control. The standard `Ownable` pattern allows direct ownership transfer via `transferOwnership(newOwner)`, which immediately grants ownership to the specified address.

```
contract AegisDFF is ERC4626, Ownable, ReentrancyGuard, IERC721Receiver, IAegisErrors, IAegisDFF {
```

If an incorrect address is provided (typo, wrong network address, or zero address), ownership is irrevocably lost. `Ownable2Step` mitigates this by requiring the new owner to explicitly accept the transfer via `acceptOwnership()`. This would make the vault's administrative capabilities more secure.

Recommendation(s): Consider replacing `Ownable` with a two-step transfer of ownership.

Status: Fixed

Update from Aegis: Will be owned by a safe that requires 4 signatures to mitigate such scenario, will change to `Ownable2Step` if we can fit it into contract size.

Update from Aegis: Added `Ownable2Step`: [bb06957451f00a0604a81990c8288f371857276e](#)

Update from Aegis: Fixed for `AegisShares` and `AegisWithdraw`: [0481902edd93f1aa37e3a1b661d2280df2c9e1d5](#)



8 Additional Notes

This section provides supplementary auditor observations regarding the code. These points were not identified as individual issues but serve as informative recommendations to enhance the overall quality and maintainability of the codebase.

- The `emergencyWithdraw(...)` function in `AegisWithdrawal.sol` uses a generic `revert()` without a specific error message when the withdrawal amount exceeds the balance. Consider using a descriptive custom error like `InsufficientBalance()` for better debugging and user experience. [1]
- In `redeemToOther(...)`, the swap output amount is calculated by measuring balance difference after the swap. The `_swapExactIn(...)` function already returns `amountOut`, so the balance measurement at line 150-151 is redundant and wastes gas. Consider using the return value directly. [2]
- In `migrateV1ToV2(...)`, the check `if (v2Variant == AegisTypes.HookVariant.None) revert HookNotAllowed()` at line 627 is redundant because the subsequent check `if (v2Variant != AegisTypes.HookVariant.V2) revert InvalidHookVariant()` already covers the `None` case. Consider removing the redundant check. [3]
- In `seedPoolFromPosition(...)`, the function checks `if (slot.baseId != 0 || slot.limitId != 0) revert ActivePositions();` to prevent seeding when Aegis V1 liquidity positions already exist for the pool. However, this check does not account for Aegis V2 positions. Consider adding a check for existing V2 liquidity position to ensure consistent behavior. [4]
- The `SIX_MONTHS` constant in `AegisShares.sol` is declared but never used in the contract. The constant `WITHDRAWAL_DELAY` (also set to 180 days) serves the same purpose. Consider removing the unused constant to reduce code clutter. [5]
- The `poolManager` and `permit2` fields in `AegisDFFRedeemLib.RedeemContext` are declared but never read within the library. Consider removing these unused struct fields to save gas on memory allocation. [6] [7]
- The `permit2` and `poolManager` fields in `AegisDFFPositionLib.RebalanceContext` are declared but never read within the library. Consider removing these unused struct fields to save gas on memory allocation. [8] [9]
- The comment at `AegisDFFPositionLib.sol:745` states that `Permit2` approvals are done in `addPool(...)`, but they are actually performed in the `approveToken(...)` function. Consider correcting the comment. [10]

Update from Aegis: Fixed: [9d3f606ec25e5631a56aa5a9ba0458e10fde3740](#)

Did not change 4, don't see a real reason to add such a check. Allows potential griefers to abuse.



9 Protocol Risks

This section outlines protocol-level risks as identified and understood by the CODESPECT audit team. These points reflect design choices, assumptions, or trust dependencies observed during the review and are documented to provide transparency regarding the protocol's risk profile from CODESPECT's perspective.

- **Centralisation Risk:** The vault is heavily reliant on two privileged roles. The `protocolOwner` controls all critical operations, including adding and removing pools, rebalancing liquidity across arbitrary tick ranges, whitelisting hooks, configuring TWAP parameters, migrating pools between hook variants, setting the Aegis Router, and collecting LP fees. The `EMERGENCY_OWNER` is able to withdraw non-operational tokens from the vault. The protocol is therefore assumed to securely manage the private keys associated with these roles.
- **TWAP Oracle Availability:** Standard redemptions without explicit slippage parameters, as well as all rebalancing operations, depend on the availability of the TWAP oracle. If the TWAP oracle becomes unavailable, for example due to insufficient observation history on a newly initialised pool or issues within the oracle mechanism, these operations will revert. This may temporarily prevent users from redeeming through the TWAP-guarded path and block administrative rebalancing. Users can bypass this limitation by using the slippage-protected redemption overload; however, the TWAP-only `redeem()` and `withdraw()` entry points would remain non-functional.
- **External Protocol Dependencies:** The vault relies on the correct operation of multiple external Uniswap V4 components, including the `PoolManager`, `PositionManager`, `Universal Router`, and `Permit2`, as well as the Aegis Engine and Aegis Router. The vault also grants these components maximum token approvals. Any vulnerability, misconfiguration, or unexpected behaviour in these external dependencies could result in partial or total loss of vault funds.



10 Evaluation of Provided Documentation

The **Aegis DFF** documentation was provided through a combination of NatSpec comments within the codebase and a separate high-level specification document:

- **Interfaces:** The `IAegisDFF` and `IAegisShares` interfaces were thoroughly documented, with comprehensive `@notice`, `@param`, and `@return` annotations covering all functions, events, and custom errors.
- **NatSpec in Core Contracts:** Documentation coverage across core contracts was uneven. `AegisDFF.sol` and `AegisWithdrawal.sol` included solid NatSpec coverage for public and external functions. In contrast, `AegisShares.sol` primarily relied on informal inline comments rather than structured NatSpec annotations.
- **External Documentation:** The `INTENDED_BEHAVIOR.md` document provided a clear high-level specification of the vault's responsibilities, the roles of individual libraries, and key system invariants. This file served as valuable architectural context during the audit.

Overall, the documentation was sufficient to support the audit process, with interface-level documentation being a particular strength. In addition, the Aegis team was consistently available to address questions and clarify design decisions raised by the CODESPECT audit team. Nevertheless, several complex areas, most notably asset valuation, fee accounting, and rebalancing logic, would benefit from more detailed inline and NatSpec documentation.

11 Test Suite Evaluation

11.1 Compilation Output

```
> forge build
[] Compiling...
[] Compiling 364 files with Solc 0.8.26
[] Solc 0.8.26 finished in 842.27s
```

11.2 Tests Output

```
> forge test
Ran 32 tests for test/foundry/new/AegisShares.t.sol:AegisSharesTest
[PASS] testFuzz_ClaimAtAnyTime(uint256) (runs: 256, : 65560, ~: 65750)
[PASS] testFuzz_PartialClaim(uint256,uint256) (runs: 256, : 70662, ~: 71373)
[PASS] test_AccumulationScenario() (gas: 167860)
[PASS] test_AdditionalDFFDeposit() (gas: 73090)
[PASS] test_AegisDffPerAegisShares() (gas: 14362)
[PASS] test_AegisDffPerSharesIncreasesAfterBurns() (gas: 64317)
[PASS] test_AgeSecondsAfterCommencement() (gas: 15354)
[PASS] test_AgeSecondsBeforeCommencement() (gas: 13198)
[PASS] test_CannotCancelAlreadyExecuted() (gas: 147228)
[PASS] test_CannotExecuteAlreadyExecuted() (gas: 147299)
[PASS] test_CannotExecuteNonQueuedWithdrawal() (gas: 20329)
[PASS] test_ClaimAtDifferentVestingStages() (gas: 65043)
[PASS] test_ClaimBeforeCommencementReverts() (gas: 12804)
[PASS] test_ClaimMoreThanBalanceReverts() (gas: 17521)
[PASS] test_ClaimToZeroAddressReverts() (gas: 14948)
[PASS] test_ClaimZeroShares() (gas: 36664)
[PASS] test_CompleteFunction() (gas: 15775)
[PASS] test_DepsNotInitializedReverts() (gas: 974621)
[PASS] test_EarlyWithdrawerGivesUpProportion() (gas: 71144)
[PASS] test_EmergencyWithdrawCancel() (gas: 129260)
[PASS] test_EmergencyWithdrawGracePeriod() (gas: 161645)
[PASS] test_EmergencyWithdrawNullAddressReverts() (gas: 14079)
[PASS] test_EmergencyWithdrawOnlyOwner() (gas: 66780)
[PASS] test_EmergencyWithdrawTimelock() (gas: 160436)
[PASS] test_FullVestingCycle() (gas: 168951)
[PASS] test_InitialDeploymentAndSetup() (gas: 59745)
[PASS] test_InitialStateBeforeCommencement() (gas: 8096)
[PASS] test_MultipleWithdrawalsAccumulation() (gas: 140242)
[PASS] test_PartialClaim() (gas: 69171)
[PASS] test_RedeemablePercentCapsAt100() (gas: 11998)
[PASS] test_UserBalances() (gas: 25499)
[PASS] test_VestingOver4Years() (gas: 22197)
Suite result: ok. 32 passed; 0 failed; 0 skipped; finished in 47.02ms (83.35ms CPU time)

Ran 6 tests for test/foundry/new/AegisDFF.SeedPoolV2.t.sol:AegisDFFSeedPoolV2Test
[PASS] test_seedPoolAmountsAndPositions() (gas: 446252)
[PASS] test_seedPoolFromPosition_transfersNFTIntoVault() (gas: 754980)
[PASS] test_seedPool_AGisToken0() (gas: 445257)
[PASS] test_seedPool_revertsIfAlreadySeeded() (gas: 411716)
[PASS] test_seedPool_revertsIfPoolNotInitialized() (gas: 659536)
[PASS] test_seedPool_tickCalculationAtBoundary() (gas: 1818837)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 141.40ms (7.35ms CPU time)

Ran 15 tests for test/foundry/new/AegisDFF.Redeem.t.sol:AegisDFFRedemptionFlowTest
[PASS] testRedeemProportionalWithFee() (gas: 38135309)
[PASS] testRedeemToAgReturnValueAccurate() (gas: 40797317)
[PASS] testRedeemToAgWithdrawsProportionally() (gas: 40811575)
[PASS] testRedeemToBlueTokenAppliesFee() (gas: 41476851)
[PASS] testRedeemToNativeCurrencyV1Hook() (gas: 2412576)
[PASS] testRedeemToOtherReturnValueAccurate() (gas: 41460899)
[PASS] testRedeemWithMinAmountOut() (gas: 40798016)
[PASS] testRedeemWithMinAmountOutRevertsOnSlippage() (gas: 40748616)
[PASS] testWithdrawHelperPropagatesSlippageGuards() (gas: 41511561)
[PASS] testWithdrawHelperRedeemFromPositionsMatchesTokenReturns() (gas: 41229747)
[PASS] testWithdrawHelperRedeemToAgMatchesBalanceChange() (gas: 41447543)
[PASS] testWithdrawHelperRedeemToOtherBluechipMatchesBalanceChange() (gas: 41740106)
[PASS] testWithdrawHelperRedeemToOtherNativeV1MatchesBalanceChange() (gas: 5562904)
```



```
[PASS] testWithdrawHelperRequiresShareAllowance() (gas: 40781848)
[PASS] testWithdrawHelperResidualDffCanBeRecovered() (gas: 41477411)
Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 2.31s (1.85s CPU time)

Ran 20 tests for test/foundry/new/AegisDFF.RedeemEngine.t.sol:AegisDFFRedemptionEngineTest
[PASS] testRedeemProportionalWithFee() (gas: 104058060)
[PASS] testRedeemProportionalWithFeeFromEngine() (gas: 104058346)
[PASS] testRedeemToAgReturnValueAccurate() (gas: 108615931)
[PASS] testRedeemToAgReturnValueAccurateFromEngine() (gas: 108615689)
[PASS] testRedeemToAgWithdrawsProportionally() (gas: 104552125)
[PASS] testRedeemToAgWithdrawsProportionallyFromEngine() (gas: 104551597)
[PASS] testRedeemToBlueTokenAppliesFee() (gas: 404524377)
[PASS] testRedeemToBlueTokenAppliesFeeFromEngine() (gas: 404523871)
[PASS] testRedeemToNativeCurrencyV1Hook() (gas: 2372936)
[PASS] testRedeemToOtherReturnValueAccurate() (gas: 404518851)
[PASS] testRedeemToOtherReturnValueAccurateFromEngine() (gas: 404519423)
[PASS] testRedeemWithMinAmountOut() (gas: 108616629)
[PASS] testRedeemWithMinAmountOutRevertsOnSlippage() (gas: 108584518)
[PASS] testWithdrawHelperPropagatesSlippageGuards() (gas: 107252398)
[PASS] testWithdrawHelperRedeemFromPositionsMatchesTokenReturns() (gas: 107156142)
[PASS] testWithdrawHelperRedeemToAgMatchesBalanceChange() (gas: 107216658)
[PASS] testWithdrawHelperRedeemToOtherBluechipMatchesBalanceChange() (gas: 107993509)
[PASS] testWithdrawHelperRedeemToOtherNativeV1MatchesBalanceChange() (gas: 595)
[PASS] testWithdrawHelperRequiresShareAllowance() (gas: 106900102)
[PASS] testWithdrawHelperResidualDffCanBeRecovered() (gas: 107246592)
Suite result: ok. 20 passed; 0 failed; 0 skipped; finished in 2.62s (7.59s CPU time)

Ran 1 test for test/foundry/new/AegisDFF.Migration.t.sol:AegisDFFMigrationTest
[PASS] test_migrateDFMToEngine() (gas: 1878680)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.00s (4.61ms CPU time)

Ran 29 tests for test/foundry/new/AegisDFF.PoolManagement.t.sol:AegisDFFPoolManagementTest
[PASS] test_addPoolRegistersEnginePool() (gas: 185640)
[PASS] test_addPoolRegistersSpotPool() (gas: 195345)
[PASS] test_addPoolRevertsWhenDuplicate() (gas: 185027)
[PASS] test_addPoolRevertsWhenHookNotAllowed() (gas: 29506)
[PASS] test_addPoolRevertsWhenMaxExceeded() (gas: 5731190)
[PASS] test_addPoolRevertsWhenNotInitialized() (gas: 646548)
[PASS] test_addPoolRevertsWhenPoolMissingAsset() (gas: 1251005)
[PASS] test_emergencyWithdraw_allowsUnrelatedToken() (gas: 697317)
[PASS] test_emergencyWithdraw_revertsForVaultAsset() (gas: 113415)
[PASS] test_emergencyWithdraw_revertsWhenTokenRegisteredInPool() (gas: 331392)
[PASS] test_getPoolInfoReturnsStoredValues() (gas: 188944)
[PASS] test_listActivePoolsReturnsIdsAndKeys() (gas: 1442878)
[PASS] test_migratePoolFromV1ToV2() (gas: 3581734)
[PASS] test_removePoolRevertsWhenPositionsActive() (gas: 503813)
[PASS] test_removePoolUpdatesIndices() (gas: 2392395)
[PASS] test_seedSwapAndRebalanceDFM() (gas: 551)
[PASS] test_seedSwapAndRebalanceEngine() (gas: 529)
[PASS] test_setRebalanceTwapMaxTicksRevertsWhenExceedingLimit() (gas: 16016)
[PASS] test_setRebalanceTwapMaxTicksUpdatesConfig() (gas: 23239)
[PASS] test_setRedeemTwapMaxTicksRevertsWhenExceedingLimit() (gas: 14234)
[PASS] test_setRedeemTwapMaxTicksUpdatesConfig() (gas: 24181)
[PASS] test_setTwapPeriodsNonOwnerReverts() (gas: 14164)
[PASS] test_setTwapPeriodsRevertsWhenExceedingLimit() (gas: 18690)
[PASS] test_setTwapPeriodsUpdatesConfig() (gas: 24163)
[PASS] test_setV1HookTogglesConfiguration() (gas: 31332)
[PASS] test_setV1HookZeroAddressReverts() (gas: 14260)
[PASS] test_setV2HookRevertsWhenEngineMissing() (gas: 292529)
[PASS] test_setV2HookTogglesConfiguration() (gas: 42551)
[PASS] test_setV2HookZeroAddressReverts() (gas: 15560)
Suite result: ok. 29 passed; 0 failed; 0 skipped; finished in 4.16s (19.28ms CPU time)

Ran 5 tests for test/foundry/new/AegisDFF.DismantlePositions.t.sol:AegisDFFDismantlePositionsTest
[PASS] test_dismantlePositions_DFMAfterRebalance() (gas: 1299967)
[PASS] test_dismantlePositions_EngineAfterRebalance() (gas: 36265783)
[PASS] test_dismantlePositions_noPositions() (gas: 29233)
[PASS] test_dismantlePositions_onlyBasePosition() (gas: 1266300)
[PASS] test_dismantlePositions_onlyOwner() (gas: 20323)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 4.25s (86.54ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  revertTo(uint256): replaced by `revertToState`
  snapshot(): replaced by `snapshotState`
Ran 5 tests for test/foundry/new/AegisDFF.SlippageProtection.t.sol:AegisDFFSlippageProtectionTest
```




```
[PASS] testFindSharesForTargetAgAmount() (gas: 18267205)
[PASS] testFindSharesForTargetBlueAmount() (gas: 21699917)
[PASS] testRedeemFromPositionsStaticCallWithSlippageProtection() (gas: 903995)
[PASS] testRedeemToAgStaticCallWithSlippageProtection() (gas: 1265686)
[PASS] testRedeemToOtherStaticCallWithSlippageProtection() (gas: 1473351)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 4.25s (138.28ms CPU time)

Ran 9 tests for test/foundry/new/AegisDFF.PoolInfo.t.sol:AegisDFFPoolInfoTest
[PASS] test_dismantlePositions_DFMAfterRebalance() (gas: 1299994)
[PASS] test_dismantlePositions_EngineAfterRebalance() (gas: 36265861)
[PASS] test_dismantlePositions_noPositions() (gas: 29255)
[PASS] test_dismantlePositions_onlyBasePosition() (gas: 1266310)
[PASS] test_dismantlePositions_onlyOwner() (gas: 20367)
[PASS] test_getPoolInfo_DFM_withBaseAndLimit() (gas: 1224016)
[PASS] test_getPoolInfo_Engine_withEngineSharesAndLimit() (gas: 36293612)
[PASS] test_getPoolInfo_bothPoolsSequentially() (gas: 37321944)
[PASS] test_getPoolInfo_seededOnly() (gas: 418367)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 4.31s (241.58ms CPU time)

Ran 5 tests for test/foundry/new/AegisDFF.ComprehensiveRebalance.t.sol:AegisDFFComprehensiveRebalanceTest
[PASS] test_comprehensiveRebalanceDFM() (gas: 4317275)
[PASS] test_comprehensiveRebalanceEngine() (gas: 4532708)
[PASS] test_migratePoolFromV1ToV2() (gas: 3590628)
[PASS] test_seedSwapAndRebalanceDFM() (gas: 1736688)
[PASS] test_seedSwapAndRebalanceEngine() (gas: 36266438)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 4.31s (128.64ms CPU time)

Ran 3 tests for test/foundry/new/AegisDFF.RebalanceFlow.t.sol:AegisDFFRebalanceFlowTest
[PASS] test_migratePoolFromV1ToV2() (gas: 3590628)
[PASS] test_seedSwapAndRebalanceDFM() (gas: 1736688)
[PASS] test_seedSwapAndRebalanceEngine() (gas: 36266438)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.31s (93.56ms CPU time)

Ran 18 tests for test/foundry/new/AegisDFF.RecentlyDeleted.t.sol:AegisDFFRecentlyDeletedTest
[PASS] test_AddPool_ClearsRecentlyDeleted() (gas: 470220)
[PASS] test_AddPool_ClearsRecentlyDeleted_Engine() (gas: 660792)
[PASS] test_DismantleWithdrawsAllEngineShares() (gas: 885997)
[PASS] test_EmergencyWithdraw_BlocksRecentlyDeletedTokens() (gas: 495860)
[PASS] test_MultipleRedemptions_ProportionalDistribution() (gas: 632167)
[PASS] test_RecentlyDeletedTokensAreReceivedOnRedemption() (gas: 554902)
[PASS] test_RecentlyDeletedTokensAreReceivedOnRedemption_Engine() (gas: 687042)
[PASS] test_RedeemFromPositions_TransfersRecentlyDeletedTokens() (gas: 555762)
[PASS] test_RedeemFromPositions_TransfersRecentlyDeletedTokens_Engine() (gas: 687182)
[PASS] test_Redeem_TransfersRecentlyDeletedTokens() (gas: 552144)
[PASS] test_Redeem_TransfersRecentlyDeletedTokens_Engine() (gas: 683283)
[PASS] test_RemovePool_AddsTokenToRecentlyDeleted() (gas: 452944)
[PASS] test_RemovePool_AddsTokenToRecentlyDeleted_Engine() (gas: 585931)
[PASS] test_RemovePool_DoesNotAddAssetToken() (gas: 446811)
[PASS] test_RemoveRecentlyDeletedToken_OnlyOwner() (gas: 457719)
[PASS] test_migratePoolFromV1ToV2() (gas: 3590661)
[PASS] test_seedSwapAndRebalanceDFM() (gas: 1742827)
[PASS] test_seedSwapAndRebalanceEngine() (gas: 36290741)
Suite result: ok. 18 passed; 0 failed; 0 skipped; finished in 4.31s (127.39ms CPU time)

Ran 12 test suites in 4.31s (39.02s CPU time): 148 tests passed, 0 failed, 0 skipped (148 total tests)
```

11.3 Notes on the Test Suite

The Aegis DFF test suite provides strong coverage of the core contract components within the audit scope. It exercises a wide range of functional flows and validates key invariants that must be preserved across normal operation.

However, several edge cases were not fully covered by the existing tests and were identified during the audit. In particular, scenarios involving native ETH pools were insufficiently exercised, and the test suite did not comprehensively explore how Uniswap V4 fees are collected, accounted for, and ultimately distributed to vault shareholders.

Expanding test coverage in these areas would improve overall robustness, reduce the risk of untested behaviour in production, and further strengthen confidence in the correctness of the codebase.